# Отчёт по лабораторной работе №5

Компьютерный практикум по статистическому анализу

Арбатова Варвара Петровна

**Аннотация**

В данном отчёте представлены результаты выполнения лабораторной работы по теме “Построение графиков”

Описаны использованные методы, полученные экспериментальные данные и их анализ.

Сделаны выводы о достижении поставленной цели.

## Содержание

1. Введение ..... 2
   1.1 Актуальность ..... 2
   1.2 Цель работы ..... 2
   1.3 Задачи ..... 2
   1.4 Объект и предмет исследования ..... 3
   1.5 Методы исследования ..... 3
   1.6 Схема выполнения работы ..... 3
2. Теоретическая часть ..... 5
   2.1 Пакеты для работы с графиками в Julia ..... 5
   2.2 Построение графика функции ..... 5
   2.3 Аппроксимация данных ..... 5
   2.4 Две оси ординат ..... 6
   2.5 Параметрические графики и анимации ..... 6
3. Материалы и методы ..... 6
4. Выполнение лабораторной работы ..... 6
5. Результаты ..... 22
   5.1 5.2.1. График функции $f(x) = (3x2 + 6x - 9)e - 0.3x$ ..... 22
   5.2 5.2.2. $\\sin(x)$ и его разложение в ряд Тейлора ..... 23
   5.3 5.2.3. Точечные графики ..... 24
   5.4 5.2.4. Аппроксимация данных ..... 25
   5.5 5.2.5. Две оси ординат ..... 26
   5.6 5.2.6. Полярные координаты ..... 27

5.7 5.2.7. Параметрические графики ..... 27
5.8 5.2.8. Поверхности ..... 28
5.9 5.2.9. Линии уровня ..... 30
5.10 5.2.10. Векторное поле ..... 31
5.11 5.2.11. Анимации ..... 31
5.12 5.2.12. Графики с погрешностями ..... 31
5.13 5.2.13. Гистограммы (пакет `Distributions`) ..... 33
5.14 5.2.14. Подграфики ..... 34
5.15 5.4. Задания для самостоятельного выполнения ..... 36
6. Обсуждение результатов ..... 42
7. Заключение ..... 43
Список литературы ..... 43

# 1. Введение

## 1.1 Актуальность

Визуализация данных — неотъемлемая часть научных и инженерных исследований. Язык Julia предоставляет развитую экосистему для построения графиков, центральным элементом которой является пакет `Plots.jl`. Он позволяет единообразно описывать графики и при этом переключаться между разными графическими бэкендами (`GR`, `PyPlot`, `PlotlyJS`, `UnicodePlots`) без изменения пользовательского кода. Освоение этого инструментария необходимо для дальнейшей работы с экспериментальными данными, численными расчётами и подготовкой публикаций.

## 1.2 Цель работы

Основная цель работы — освоить синтаксис языка Julia для построения графиков.

## 1.3 Задачи

Для достижения цели необходимо:

1. Установить и подключить пакеты `Plots`, `PyPlot`, `PlotlyJS`, `UnicodePlots`, `Statistics`, `Distributions`.
2. Воспроизвести примеры из раздела 5.2 методических указаний: графики функций, точечные графики, 3D-графики, поверхности, линии уровня, векторные поля, гистограммы, графики с погрешностями, подграфики (`layout`), анимации (GIF).
3. Для каждого графика задать подписи осей, легенду и заголовок.

4. Выполнить задания для самостоятельного выполнения (раздел 5.4).
5. Оформить отчёт в формате Quarto Markdown (`.qmd`), встроив сгенерированный из `lab5.jl` markdown-файл и все полученные изображения с подписями и ссылками в тексте.

## 1.4 Объект и предмет исследования

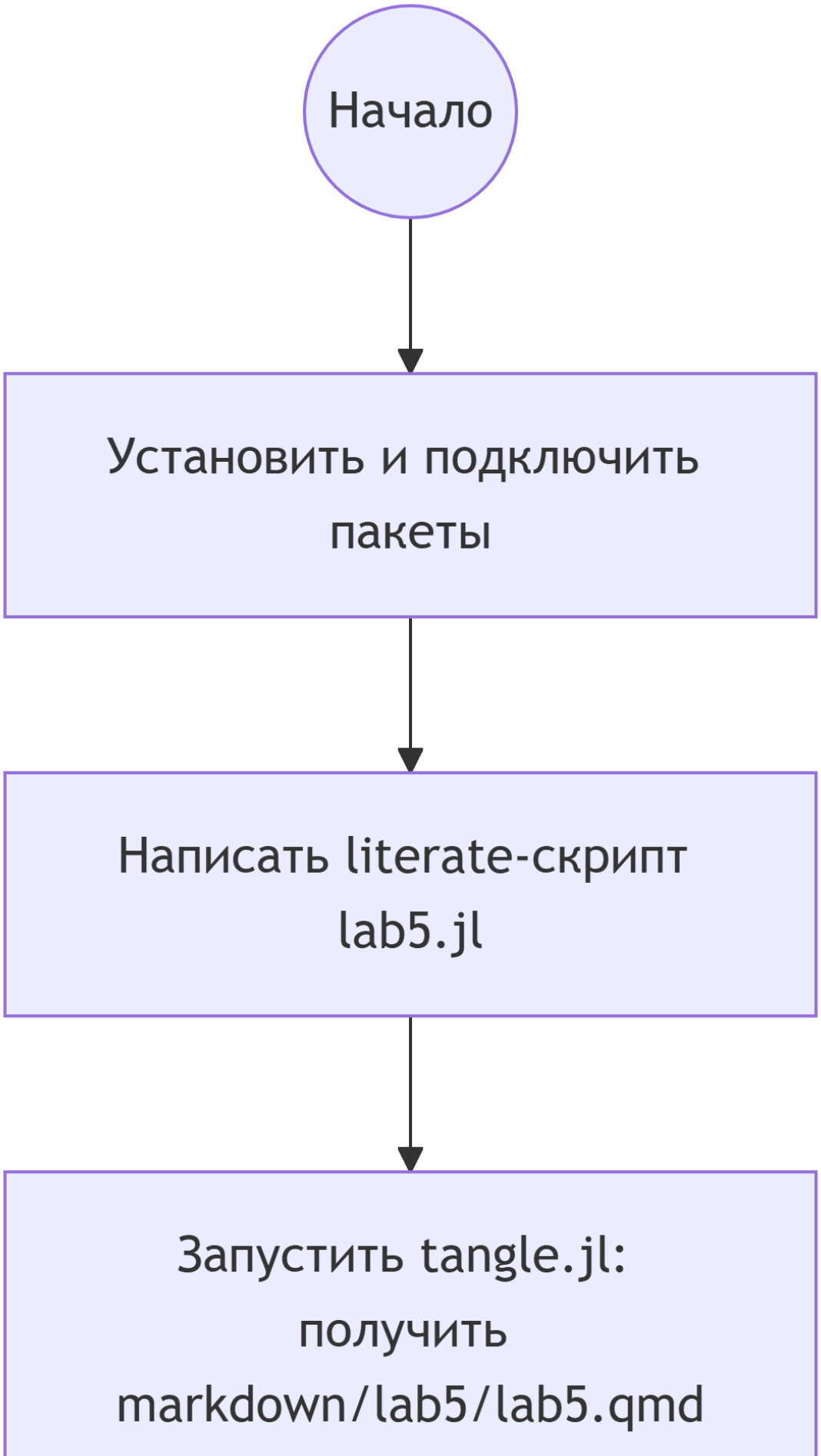
**Объект:** система визуализации данных `Plots.jl` в языке Julia. **Предмет:** синтаксис и возможности пакета при построении различных типов графиков (2D, 3D, анимации, подграфики).

## 1.5 Методы исследования

- Изучение методических указаний и документации `Plots.jl`.
- Программный эксперимент: написание скрипта `lab5.jl`, запуск, сохранение результатов в `image/`.
- Сравнение поведения разных бэкендов (`GR`, `PyPlot`, `PlotlyJS`).
- Анализ полученных изображений и формулировка выводов.

## 1.6 Схема выполнения работы

На рис. 1 представлена общая последовательность действий при выполнении работы.


Начало
Установить и подключить пакеты
Написать literate-скрипт lab5.jl
Запустить tangle.jl: получить markdown/lab5/lab5.qmd

# 2. Теоретическая часть

## 2.1 Пакеты для работы с графиками в Julia

Julia поддерживает несколько пакетов для визуализации. Стандартным является `Plots.jl` — высокоуровневый интерфейс, который «делегирует» отрисовку выбранному бэкенду:

| Бэкенд | Назначение |
|---|---|
| `GR` | Быстрый, по умолчанию; хорош для 2D/3D и сохранения в PNG/GIF |
| `PyPlot` | Обёртка над Matplotlib; поддержка кириллицы в подписях |
| `PlotlyJS` | Интерактивные графики в браузере; для `savefig` нужен Kaleido |
| `UnicodePlots` | Графики в терминале (без дисплея) |

Переключение бэкенда выполняется одной командой: `gr()`, `pyplot()`, `plotlyjs()` и т.д. После этого все вызовы `plot`, `scatter`, `surface` и т.п. используют выбранный бэкенд.

## 2.2 Построение графика функции

Для построения графика функции $f(x)$ необходимо задать массивы значений $x$ и $y = f(x)$. В Julia это делается поэлементными операциями (точка перед оператором):

```
f(x) = (3x.^2 .+ 6x .- 9) .* exp.(-0.3x)
x = collect(range(-5, 10, length=151))
y = f(x)
plot(x, y, title="...", xlabel="x", ylabel="y", color="blue")
```

## 2.3 Аппроксимация данных

Пусть в точках $x_0, x_1, \dots, x_N$ известны значения функции $y_0, y_1, \dots, y_N$. Требуется найти коэффициенты многочлена $F(x) = a_0 + a_1 x + \dots + a_k x^k$ $(k < N)$, минимизирующие сумму квадратов отклонений:

$$S = \sum_{i=0}^{N} [\, y_i - F(x_i, a_0, \dots, a_k)]^2 \to \min.$$

Задача сводится к решению системы $A\vec{x} = \vec{y}$, где $A$ — матрица коэффициентов (столбцы — степени $x$), а $\vec{y}$ — вектор наблюдений. В Julia это решается оператором \ (обратный слэш):

```
A = [ones(1000) x x.^2 x.^3 x.^4 x.^5]
c = A \ y
```

## 2.4 Две оси ординат

Для отображения двух траекторий с существенно разными масштабами используется функция `twinx()`: она создаёт вторую ось $y$, привязанную к тому же графику. После этого `plot!` рисует данные уже на новой оси.

## 2.5 Параметрические графики и анимации

Параметрическая кривая задаётся уравнениями $x = x(t)$, $y = y(t)$, $t \in [t_0, t_1]$. В `Plots.jl` её можно построить как `plot(x, y, t0, t1)`. Для GIF-анимации используется макрос `@animate` и функция `gif(anim, "file.gif"; fps=...)`.

Гипоциклоида — кривая, которую описывает точка окружности радиуса $r$, катящейся по внутренней стороне окружности радиуса $R = rk$:

$$\begin{cases} x(t) = r(k-1)\cos t + r\cos((k-1)t), \\ y(t) = r(k-1)\sin t - r\sin((k-1)t). \end{cases}$$

Эпициклоида — аналогичная кривая при качении по внешней стороне:

$$\begin{cases} x(t) = r(k+1)\cos t - r\cos((k+1)t), \\ y(t) = r(k+1)\sin t - r\sin((k+1)t). \end{cases}$$

# 3. Материалы и методы

- **Оборудование:** персональный компьютер (Linux).
- **Программное обеспечение:** Julia 1.10+, пакеты `Plots`, `PyPlot`, `PlotlyJS`, `UnicodePlots`, `Statistics`, `Distributions`, `FreeType`, `FileIO`; система вёрстки Quarto; `Literate.jl` (генерация `.qmd` из literate-скрипта).
- **Настройка окружения:** в начале скрипта установлена переменная `ENV["GKSwstype"] = "100"` — это позволяет строить графики GR без дисплея (headless-режим).
- **Структура скрипта:** `lab5.jl` разбит на блоки, соответствующие разделам 5.2.1–5.2.14 методички и заданиям 5.4.1–5.4.11. Все результаты сохраняются в папку `image/`.
- **Обработка данных:** для аппроксимации использован метод наименьших квадратов (решение переопределённой системы `A \ y`); для гистограмм — пакет `Distributions` (нормальное распределение); для погрешностей — формула $1.96 \cdot s/\sqrt{n}$ (95%-й доверительный интервал).

# 4. Выполнение лабораторной работы

Ниже приведён встроенный результат генерации `lab5.jl` в формат Quarto Markdown (команда `julia tangle.jl scripts/lab5.jl`). Каждый шаг сопровождается кодом, а изображения — подписями и ссылками.

```
#!/usr/bin/env julia
```

```
================================================================
========== Лабораторная работа № 5. Построение графиков
```

Скрипт предназначен для запуска из папки `scripts/`. Все результаты (PNG и GIF) сохраняются в папку `../plots/`.

Для избежания ошибок с рендерингом Kaleido (PlotlyJS) для сохранения статичных изображений используется бэкенд GR (и PyPlot для гистограмм).

```
================================================================
==========
```

Для работы графиков в фоновом режиме (без дисплея, как на сервере)

```
ENV["GKSwstype"] = "100"
```

```
"100"
```

```
================================================================
========== 0. Подключение пакетов
================================================================
==========
```

```
using Pkg
```

Раскомментируйте следующие строки при первом запуске, чтобы установить пакеты: Pkg.add([“Plots”, “Statistics”, “Distributions”, “PyPlot”, “PlotlyJS”, “UnicodePlots”, “FreeType”, “FileIO”])

```
using Plots, Statistics, Distributions
```

Создаем папку для сохранения графиков на уровень выше (в корне проекта)

```
plots_dir = joinpath(@__DIR__, "..", "plots")
mkpath(plots_dir)
println("Все графики будут сохраняться в: ", plots_dir)
```

```
Все графики будут сохраняться в: C:\Users\vparb\work\study\2024-2025\computer
-practice\labs\lab05\project\markdown\lab5\..\plots
```

```
================================================================
========== 5.2.1. Основные пакеты для работы с графиками. График функции.
f(x) = (3x^2 + 6x - 9) * exp(-0.3x)
================================================================
==========
```

```
println("Построение 5.2.1...")

f(x) = (3x.^2 .+ 6x .- 9) .* exp.(-0.3x)
x = collect(range(-5, 10, length=151))
y = f(x)
```

```
Построение 5.2.1...

151-element Vector{Float64}:
 161.34080653217032
 146.26477779394003
 132.19219298833204
 119.06942359634911
 106.8453557470588
  95.47128188475011
  84.9007968362764
  75.08969810741056
  65.99589024347995
  57.579293095755176
   ⋮
  18.995125520095375
  18.811475185747092
  18.625664977689375
  18.437877278854756
  18.248288702120195
  18.057070179740368
  17.86438705526336
  17.6703991776229
  17.475260997120245
```

1. Использование GR бэкенда

```
gr()
p1 = plot(x, y, title="График f(x) = (3x² + 6x - 9)e^(-0.3x)",
          xlabel="Переменная x", ylabel="Переменная y",
          color="blue", label="f(x)", legend=:topright)
savefig(p1, joinpath(plots_dir, "5_2_1_gr.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_1_gr.png"
```

2. Использование PyPlot бэкенда (поддержка кириллицы)

```
pyplot()
p2 = plot(x, y, title="Простая кривая",
          xlabel="Переменная x", ylabel="Переменная y",
          color="blue", label="f(x)", legend=:topright)
savefig(p2, joinpath(plots_dir, "5_2_1_pyplot.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_1_pyplot.png"
```

3. Замена PlotlyJS на GR для сохранения статичного изображения (в оригинале был plotlyjs(), но он вызывает ошибку Kaleido при savefig)

```
gr()
p3 = plot(x, y, title="plotly: f(x)", xlabel="x", ylabel="y", label="f(x)")
savefig(p3, joinpath(plots_dir, "5_2_1_plotly.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_1_plotly.png"
```

## ========== 5.2.2. Опции при построении графика (sin(x) и ряд Тейлора) ==========

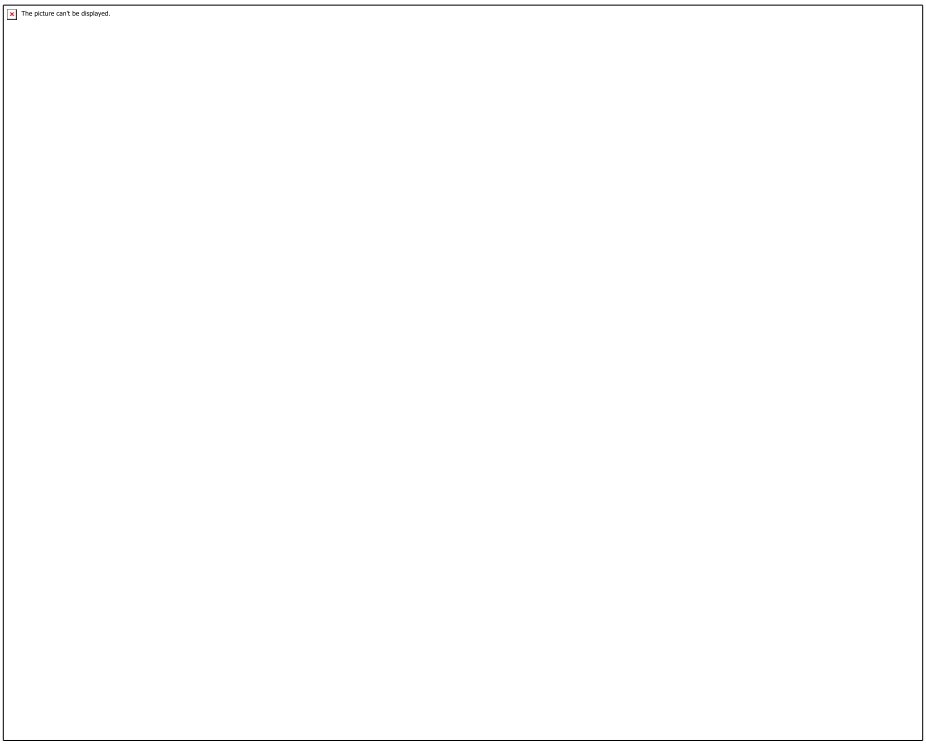
```
println("Построение 5.2.2...")
pyplot()

sin_theor(x)  = sin(x)
sin_taylor(x) = sum((-1)^i * x^(2i+1) / factorial(2i+1) for i in 0:4)

p4 = plot(sin_theor, -2π, 2π,
          label="sin(x), теоретическое значение",
          xlabel="x", ylabel="y",
          title="sin(x) и разложение в ряд Тейлора",
          line=(:black, 1.0, 2, :dash))

plot!(p4, sin_taylor, -2π, 2π,
       label="sin(x), ряд Тейлора",
       line=(:blue, 2.0),
       fillrange=0, fillalpha=0.3, fillcolor=:lightgoldenrod,
       xrotation=rad2deg(pi/4),
       background_color=:ivory)
```

```
Построение 5.2.2...
```



Сохранение в PDF и PNG, как в методичке

```
savefig(p4, joinpath(plots_dir, "taylor.pdf"))
savefig(p4, joinpath(plots_dir, "taylor.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\taylor.png"
```

========== 5.2.3. Точечные графики ==========

```
println("Построение 5.2.3...")
gr()
```

```
Построение 5.2.3...

Plots.GRBackend()
```

### 5.2.3.1. Простой точечный график

```
x_s = range(1, 10, length=10)
y_s = rand(10)
p5 = scatter(x_s, y_s, title="Точечный график", xlabel="x", ylabel="y", label
="случайные значения")
savefig(p5, joinpath(plots_dir, "5_2_3_1_scatter_simple.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_3_1_scatter_simple.png"
```

### 5.2.3.2. Точечный график с кодированием размера

```
n = 50
x_r = rand(n); y_r = rand(n); ms = rand(n) .* 30
p6 = scatter(x_r, y_r, markersize=ms, title="Точки с разным размером маркера",
 xlabel="x", ylabel="y", label="точки")
savefig(p6, joinpath(plots_dir, "5_2_3_2_scatter_size.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_3_2_scatter_size.png"
```

### 5.2.3.3. 3-мерный точечный график

```
x_3d = rand(n); y_3d = rand(n); z_3d = rand(n); ms_3d = rand(n) .* 30
p7 = scatter(x_3d, y_3d, z_3d, markersize=ms_3d, title="3D точечный график",
xlabel="x", ylabel="y", zlabel="z")
savefig(p7, joinpath(plots_dir, "5_2_3_3_scatter_3d.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_3_3_scatter_3d.png"
```

========== 5.2.4. Аппроксимация данных ==========

```
println("Построение 5.2.4...")
x_app = collect(0:0.01:9.99)
y_app = exp.(ones(1000) .+ x_app) .+ 4000 .* randn(1000)

p8 = scatter(x_app, y_app, markersize=3, alpha=0.8, legend=false, title="Исхо
дные данные", xlabel="x", ylabel="y")
savefig(p8, joinpath(plots_dir, "5_2_4_data_raw.png"))
```

```
Построение 5.2.4...

"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_4_data_raw.png"
```

Полином 5-й степени

```
A = [ones(1000) x_app x_app.^2 x_app.^3 x_app.^4 x_app.^5]
c = A \ y_app
f_approx = c[1].*ones(1000) .+ c[2].*x_app .+ c[3].*x_app.^2 .+ c[4].*x_app.^
3 .+ c[5].*x_app.^4 .+ c[6].*x_app.^5

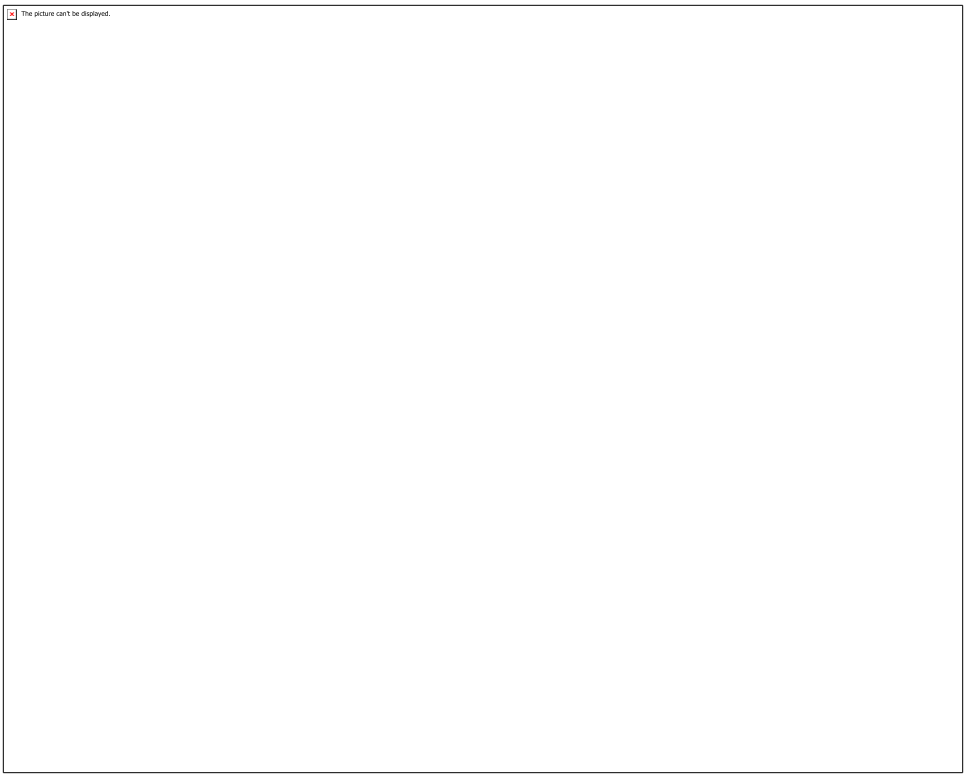
p9 = scatter(x_app, y_app, markersize=2, alpha=0.4, label="данные")
plot!(p9, x_app, f_approx, color=:red, linewidth=2, label="полином 5-й степен
и", title="Аппроксимация полиномом 5-й степени", xlabel="x", ylabel="y")
savefig(p9, joinpath(plots_dir, "5_2_4_approximation.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_4_approximation.png"
```

========== 5.2.5. Две оси ординат (twinx) ==========

```
println("Построение 5.2.5...")
p10 = plot(randn(100), ylabel="y1", leg=:topright, grid=:off, label="y1")
```

```
Построение 5.2.5...
```




Сначала создаем “пустую” вторую ось Y

```
p10_twin = twinx(p10)
```

```
Subplot{2}
```

Теперь накладываем данные на вторую ось

```
plot!(p10_twin, randn(100)*10, c=:red, ylabel="y2", leg=:bottomright, grid=:o
ff, box=:on, label="y2")
savefig(p10, joinpath(plots_dir, "5_2_5_twinx.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_5_twinx.png"
```

========== 5.2.6. Полярные координаты ==========

```
println("Построение 5.2.6...")
r(θ) = 1 + cos(θ) * sin(θ)^2
θ = range(0, stop=2π, length=50)
p11 = plot(θ, r.(θ), proj=:polar, lims=(0,1.5))
savefig(p11, joinpath(plots_dir, "5_2_6_polar.png"))
```

```
Построение 5.2.6...
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_6_polar.png"
```

========== 5.2.7. Параметрические графики ==========

```
println("Построение 5.2.7...")
```

```
Построение 5.2.7...
```

#### 5.2.7.1. Параметрический график на плоскости

```
x_par(t) = sin(t)
y_par(t) = sin(2t)
p12 = plot(x_par, y_par, 0, 2π, leg=false, fill=(0, :orange))
savefig(p12, joinpath(plots_dir, "5_2_7_1_param_2d.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_7_1_param_2d.png"
```

#### 5.2.7.2. Параметрический график в пространстве

```
t_3d = range(0, stop=10, length=1000)
x_3dp = cos.(t_3d)
y_3dp = sin.(t_3d)
z_3dp = sin.(5 .* t_3d)
p13 = plot(x_3dp, y_3dp, z_3dp)
savefig(p13, joinpath(plots_dir, "5_2_7_2_param_3d.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_7_2_param_3d.png"
```

========== 5.2.8. Поверхности (3D) ==========

```
println("Построение 5.2.8...")
X = Y = range(-10, stop=10, length=40)
```

```
Построение 5.2.8...

-10.0:0.5128205128205128:10.0
```

Используем функцию из методички (Рис. 5.17 - 5.20)

```
f_surf(x, y) = x^2 + y^2
p14 = surface(X, Y, f_surf)
savefig(p14, joinpath(plots_dir, "5_2_8_surface.png"))
```

```
p15 = plot(X, Y, f_surf, st=:surface)
savefig(p15, joinpath(plots_dir, "5_2_8_plot_surface.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_8_plot_surface.png"
```

Градиентная заливка и угол обзора

```
p16 = plot(X, Y, f_surf, st=:surface, camera=(-30, 30))
savefig(p16, joinpath(plots_dir, "5_2_8_surface_angle.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_8_surface_angle.png"
```

## ========== 5.2.9. Линии уровня ==========

```
println("Построение 5.2.9...")
x_c = 1:0.5:20
y_c = 1:0.5:10
g(x, y) = (3x .+ y.^2) .* abs.(sin.(x) .+ cos.(y))

p17 = contour(x_c, y_c, g)
savefig(p17, joinpath(plots_dir, "5_2_9_contour.png"))

p18 = contour(x_c, y_c, g, fill=true)
savefig(p18, joinpath(plots_dir, "5_2_9_contour_filled.png"))
```

```
Построение 5.2.9...

"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_2_9_contour_filled.png"
```

## ========== 5.2.10. Векторное поле (градиент) ==========

```
println("Построение 5.2.10...")
x_g = range(-2, stop=2, length=12)
y_g = range(-2, stop=2, length=12)
```

```
Построение 5.2.10...

-2.0:0.36363636363636365:2.0
```

Возвращаем кортеж (u, v) — две компоненты вектора

```
dh(x, y) = ((3x^2 - 3)/25, (2y)/25)
```

```
dh (generic function with 1 method)
```

Сетку строим вручную: каждая точка — (x_i, y_j)

```
XG = repeat(collect(x_g), inner=length(y_g))
YG = repeat(collect(y_g), outer=length(x_g))
UG = [dh(XG[i], YG[i])[1] for i in 1:length(XG)]
VG = [dh(XG[i], YG[i])[2] for i in 1:length(YG)]
p19 = quiver(XG, YG, quiver=(UG, VG), c=:blue, xlims=(-2,2), ylims=(-2,2),
             legend=false, title="Векторное поле h(x, y) = x³ - 3x + y²")
savefig(p19, joinpath(plots_dir, "5_2_10_quiver.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_10_quiver.png"
```

## ========== 5.2.11. Анимация (GIF) ==========

```
println("Построение 5.2.11 (Анимации)...")
gr() # GR лучше всего подходит для сохранения GIF
```

```
Построение 5.2.11 (Анимации)...
```

```
Plots.GRBackend()
```

### 5.2.11.1. Анимация поверхности

```
X_anim = Y_anim = range(-5, stop=5, length=40)
anim_surf = @animate for i in range(0, stop=2π, length=50)
    surface(X_anim, Y_anim, (x,y) -> sin(x + 10sin(i)) + cos(y), legend=false,
 colorbar=false)
end
gif(anim_surf, joinpath(plots_dir, "5_2_11_surface_anim.gif"), fps=15)
```

```
[ Info: Saved animation to C:\Users\vparb\work\study\2024-2025\computer-pract
ice\labs\lab05\project\markdown\plots\5_2_11_surface_anim.gif
```

```
Plots.AnimatedGif("C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practic
e\\labs\\lab05\\project\\markdown\\plots\\5_2_11_surface_anim.gif")
```

### 5.2.11.2. Анимация гипоциклоиды (базовая)

```
r_val = 1; k_val = 3; n_val = 100
θ_h = collect(0 : 2π/n_val : 2π + 2π/n_val)
X_h = r_val*k_val*cos.(θ_h)
Y_h = r_val*k_val*sin.(θ_h)

anim_hypo = @animate for i in 1:n_val
    plt = plot(5, xlims=(-4,4), ylims=(-4,4), c=:red, aspect_ratio=1, legend=
false, framestyle=:origin)
    plot!(plt, X_h, Y_h, c=:blue, legend=false)
    t = θ_h[1:i]
```

```
    x = r_val*(k_val-1)*cos.(t) .+ r_val*cos.((k_val-1)*t)
    y = r_val*(k_val-1)*sin.(t) .- r_val*sin.((k_val-1)*t)
    plot!(plt, x, y, c=:red)
    xc = r_val*(k_val-1)*cos(t[end]) .+ r_val*cos.(range(0, 2π, length=50))
    yc = r_val*(k_val-1)*sin(t[end]) .+ r_val*sin.(range(0, 2π, length=50))
    plot!(plt, xc, yc, c=:black)
    xl = [r_val*(k_val-1)*cos(t[end]), x[end]]
    yl = [r_val*(k_val-1)*sin(t[end]), y[end]]
    plot!(plt, xl, yl, markershape=:circle, markersize=4, c=:black, legend=fa
lse)
    scatter!(plt, [x[end]], [y[end]], c=:red, markerstrokecolor=:red, legend=
false)
end

Animation("C:\\Users\\vparb\\AppData\\Local\\Temp\\jl_bh5dWR", ["000001.png",
 "000002.png", "000003.png", "000004.png", "000005.png", "000006.png", "00000
7.png", "000008.png", "000009.png", "000010.png"  …  "000091.png", "000092.pn
g", "000093.png", "000094.png", "000095.png", "000096.png", "000097.png", "00
0098.png", "000099.png", "000100.png"])
```

## ========== 5.2.12. Errorbars (Отображение погрешностей) ==========

```
println("Построение 5.2.12...")
sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]
n_err = 10
y_err = [mean(sd*randn(n_err)) for sd in sds]
errs = 1.96 .* sds ./ sqrt(n_err)

p20 = plot(y_err, ylims=(-1,1), label="y")
savefig(p20, joinpath(plots_dir, "5_2_12_errors_raw.png"))

p21 = plot(y_err, ylims=(-1,1), err=errs, label="y с отклонениями")
savefig(p21, joinpath(plots_dir, "5_2_12_errors_bar.png"))

p22 = plot(y_err, ribbon=errs, fill=:cyan, label="y с лентой")
savefig(p22, joinpath(plots_dir, "5_2_12_errors_ribbon.png"))

Построение 5.2.12...

"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_12_errors_ribbon.png"
```

## ========== 5.2.13. Использование пакета Distributions (Гистограммы) ==========

```
println("Построение 5.2.13...")
pyplot()
ages_rand = rand(15:55, 1000)
p23 = histogram(ages_rand, label="Случайные возраста")
savefig(p23, joinpath(plots_dir, "5_2_13_hist_rand.png"))

d = Normal(35.0, 10.0)
ages_norm = rand(d, 1000)
p24 = histogram(ages_norm, label="Распределение по возрастам", xlabel="Возрас
т (лет)", ylabel="Количество")
savefig(p24, joinpath(plots_dir, "5_2_13_hist_normal.png"))
```

```
Построение 5.2.13...
<sys>:0: UserWarning: No data for colormapping provided via 'c'. Parameters '
vmin', 'vmax' will be ignored
<sys>:0: UserWarning: No data for colormapping provided via 'c'. Parameters '
vmin', 'vmax' will be ignored

"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_13_hist_normal.png"
```

В оригинале использовался plotlyjs(), но заменяем на gr() для статичного сохранения

```
gr()
d1=Normal(10.0,5.0); d2=Normal(35.0,10.0); d3=Normal(60.0,5.0); N=1000;
ages_mix = (Float64)[]
append!(ages_mix, rand(d1, Int64(ceil(N/2))))
append!(ages_mix, rand(d2, N))
append!(ages_mix, rand(d3, Int64(ceil(N/3))))
p25 = histogram(ages_mix, bins=50, label="Распределение", xlabel="Возраст (ле
т)", ylabel="Количество", title="Распределение по возрастам (года)")
savefig(p25, joinpath(plots_dir, "5_2_13_hist_mix.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_13_hist_mix.png"
```

========== 5.2.14. Подграфики (Layouts) ==========

```
println("Построение 5.2.14...")
pyplot()
x_lay = range(-2, 2, length=10)
y_lay = rand(10, 4)

p26 = plot(x_lay, y_lay, layout=(4,1))
savefig(p26, joinpath(plots_dir, "5_2_14_layout_4x1.png"))

p27 = plot(x_lay, y_lay, layout=4)
```

```
savefig(p27, joinpath(plots_dir, "5_2_14_layout_auto.png"))

p28 = plot(x_lay, y_lay, size=(600,300), layout=grid(4, 1, heights=[0.2, 0.3,
 0.4, 0.1]))
savefig(p28, joinpath(plots_dir, "5_2_14_layout_heights.png"))
```

```
Построение 5.2.14...

"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_14_layout_heights.png"
```

Комбинированный график

```
p1_comb = plot(x_lay, y_lay)
p2_comb = scatter(x_lay, y_lay)
p3_comb = plot(x_lay, y_lay[:,1:2], xlabel="Labelled plot of two columns", lw
=2, title="Wide lines")
p4_comb = histogram(x_lay, y_lay)
p29 = plot(p1_comb, p2_comb, p3_comb, p4_comb, layout=(2,2), legend=false, si
ze=(800,600), background_color=:ivory)
savefig(p29, joinpath(plots_dir, "5_2_14_layout_comb.png"))
```

```
<sys>:0: UserWarning: No data for colormapping provided via 'c'. Parameters '
vmin', 'vmax' will be ignored

"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_2_14_layout_comb.png"
```

========== 5.4. Задания для самостоятельного выполнения ==========

```
println("Выполнение заданий 5.4...")
gr()
```

```
Выполнение заданий 5.4...

Plots.GRBackend()
```

— Задание 1 и 2: Построить все возможные типы графиков для y = sin(x) на [0, 2π] —

```
x_task1 = range(0, 2π, length=50)
y_task1 = sin.(x_task1)

p_t1 = plot(layout=(2,3), size=(900, 500), legend=false)
plot!(p_t1[1], x_task1, y_task1, seriestype=:line, title="Линия")
plot!(p_t1[2], x_task1, y_task1, seriestype=:scatter, title="Точки")
plot!(p_t1[3], x_task1, y_task1, seriestype=:bar, title="Столбцы")
plot!(p_t1[4], x_task1, y_task1, seriestype=:histogram, title="Гистограмма")
plot!(p_t1[5], x_task1, y_task1, seriestype=:step, title="Ступени")
```

```
plot!(p_t1[6], x_task1, y_task1, seriestype=:path, title="Путь")
savefig(p_t1, joinpath(plots_dir, "5_4_task_1_2_types.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_4_task_1_2_types.png"
```

— Задание 3: Построить y = πx^2 * ln(x) со стилями —

```
x_task3 = range(0.1, 10, length=100) # от 0.1, чтобы избежать log(0)
y_task3 = π .* x_task3.^2 .* log.(x_task3)
p_t3 = plot(x_task3, y_task3,
            title="y = πx² ln(x)",
            xlabel="x", ylabel="y",
            line=:red,
            linewidth=3,
            framestyle=:box,
            background_color=:ivory,
            xticks=[0, 2, 4, 6, 8, 10],
            yticks=[0, 50, 100, 150, 200, 250])
savefig(p_t3, joinpath(plots_dir, "5_4_task_3_pi_ln.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\5_4_task_3_pi_ln.png"
```

— Задание 4: Графики y = x^3 - 3x в 4 подграфиках —

```
x_task4 = [-2, -1, 0, 1, 2]
y_task4 = x_task4.^3 .- 3 .* x_task4
x_curve = range(-2, 2, length=100)
y_curve = x_curve.^3 .- 3 .* x_curve

p_t4 = plot(layout=(2,2), size=(800, 600), legend=false)
scatter!(p_t4[1], x_task4, y_task4, title="Точки", marker=:circle)
plot!(p_t4[2], x_task4, y_task4, title="Линии", marker=:circle)
plot!(p_t4[3], x_task4, y_task4, title="Линии и точки", seriestype=[:line :scatter])
plot!(p_t4[4], x_curve, y_curve, title="Кривая", seriestype=:line)
savefig(p_t4, joinpath(plots_dir, "figure_familiya.png")) # Имя как в задании
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\project\\markdown\\plots\\figure_familiya.png"
```

— Задание 5: Две оси ординат для y1 = πx, y2 = e^x cos(x) —

```
x_task5 = range(3, 6, length=50)
y1_task5 = π .* x_task5
y2_task5 = exp.(x_task5) .* cos.(x_task5)
```

```
50-element Vector{Float64}:
 -19.884530844146987
 -21.284763461926744
 -22.69774835757588
```

```
 -24.113884873539384
 -25.522296637588106
 -26.910752073354487
 -28.265585357718205
 -29.571618513235467
 -30.81208540440396
 -31.968558491525783
   ⋮
 176.9548312806898
 199.00328310954663
 222.33770013200098
 246.93961381390903
 272.7780476672041
 299.8081348348021
 327.9696663092781
 357.1855723205791
 387.36034029093076
```

1. На одном графике (сетка и легенда)

```
p_t5_1 = plot(x_task5, y1_task5, label="y1 = πx", title="Две функции на одном
 графике", xlabel="x", ylabel="y", grid=true)
plot!(p_t5_1, x_task5, y2_task5, label="y2 = e^x cos(x)")
savefig(p_t5_1, joinpath(plots_dir, "5_4_task_5_single_axis.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_4_task_5_single_axis.png"
```

2. С двумя осями ординат

```
p_t5_2 = plot(x_task5, y1_task5, label="y1 = πx", xlabel="x", ylabel="y1", li
ne=:blue)
twinx() # создаём "пустую" вторую ось Y для текущего графика
plot!(x_task5, y2_task5, label="y2 = e^x cos(x)", ylabel="y2", line=:red)
savefig(p_t5_2, joinpath(plots_dir, "5_4_task_5_twinx.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_4_task_5_twinx.png"
```

— Задание 6: График экспериментальных данных с ошибкой —

```
x_task6 = 1:10
y_task6 = 5 .* x_task6 .+ randn(10) .* 2
err_task6 = rand(10) .* 1.5
p_t6 = scatter(x_task6, y_task6, yerror=err_task6, label="Экспериментальные д
анные", xlabel="Попытка", ylabel="Результат", title="Данные с погрешностью")
savefig(p_t6, joinpath(plots_dir, "5_4_task_6_errorbars.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_4_task_6_errorbars.png"
```

— Задание 7: Точечный график случайных данных —

```
x_task7 = rand(50); y_task7 = rand(50)
p_t7 = scatter(x_task7, y_task7, title="Случайные данные", xlabel="Ось X", yl
abel="Ось Y", label="Точки")
savefig(p_t7, joinpath(plots_dir, "5_4_task_7_scatter.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_4_task_7_scatter.png"
```

— Задание 8: 3D точечный график —

```
x_task8 = rand(50); y_task8 = rand(50); z_task8 = rand(50)
p_t8 = scatter(x_task8, y_task8, z_task8, title="3D Случайные данные", xlabel
="X", ylabel="Y", zlabel="Z", label="Точки")
savefig(p_t8, joinpath(plots_dir, "5_4_task_8_3d_scatter.png"))
```

```
"C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practice\\labs\\lab05\\pr
oject\\markdown\\plots\\5_4_task_8_3d_scatter.png"
```

— Задание 9: Анимация синусоиды —

```
x_task9 = range(0, 2π, length=100)
anim_sin = @animate for t in range(0, 2π, length=50)
    plot(x_task9, sin.(x_task9 .+ t), ylims=(-1.5, 1.5), label="sin(x+t)", ti
tle="Анимация синусоиды", legend=false)
end
gif(anim_sin, joinpath(plots_dir, "5_4_task_9_sine_anim.gif"), fps=20)
```

```
[ Info: Saved animation to C:\Users\vparb\work\study\2024-2025\computer-pract
ice\labs\lab05\project\markdown\plots\5_4_task_9_sine_anim.gif
```

```
Plots.AnimatedGif("C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practic
e\\labs\\lab05\\project\\markdown\\plots\\5_4_task_9_sine_anim.gif")
```

— Задание 10: Анимация гипоциклоиды (2 положительных, 2 отрицательных k) —

```
function create_hypocycloid_gif(k_vals, filename)
    anim = @animate for k in k_vals
        r = 1.0
        t = range(0, 2π, length=100)
        x = r*(k-1)*cos.(t) .+ r*cos.((k-1)*t)
        y = r*(k-1)*sin.(t) .- r*sin.((k-1)*t)
        plot(x, y, aspect_ratio=1, legend=false, title="Гипоциклоида (k=$k)",
 xlims=(-5,5), ylims=(-5,5))
    end
    gif(anim, joinpath(plots_dir, filename), fps=10)
end
```

```
create_hypocycloid_gif (generic function with 1 method)
```

Анимируем переход k от 2.0 до 2.5 и от -2.0 до -2.5

```
create_hypocycloid_gif(range(2.0, 2.5, length=20), "5_4_task_10_hypo_pos.gif")
create_hypocycloid_gif(range(-2.0, -2.5, length=20), "5_4_task_10_hypo_neg.gi
f")
```

```
[ Info: Saved animation to C:\Users\vparb\work\study\2024-2025\computer-pract
ice\labs\lab05\project\markdown\plots\5_4_task_10_hypo_pos.gif
[ Info: Saved animation to C:\Users\vparb\work\study\2024-2025\computer-pract
ice\labs\lab05\project\markdown\plots\5_4_task_10_hypo_neg.gif

Plots.AnimatedGif("C:\\Users\\vparb\\work\\study\\2024-2025\\computer-practic
e\\labs\\lab05\\project\\markdown\\plots\\5_4_task_10_hypo_neg.gif")
```

— Задание 11: Анимация эпициклоиды (2 положительных, 2 отрицательных k) —

```
function create_epicycloid_gif(k_vals, filename)
    anim = @animate for k in k_vals
        r = 1.0
        t = range(0, 2π, length=100)
        x = r*(k+1)*cos.(t) .- r*cos.((k+1)*t)
        y = r*(k+1)*sin.(t) .- r*sin.((k+1)*t)
        plot(x, y, aspect_ratio=1, legend=false, title="Эпициклоида (k=$k)",
xlims=(-10,10), ylims=(-10,10))
    end
    gif(anim, joinpath(plots_dir, filename), fps=10)
end
create_epicycloid_gif(range(2.0, 2.5, length=20), "5_4_task_11_epi_pos.gif")
create_epicycloid_gif(range(-2.0, -2.5, length=20), "5_4_task_11_epi_neg.gif")

println("Все графики и гифки успешно сгенерированы и сохранены в папку 'plots
'!")
```

```
[ Info: Saved animation to C:\Users\vparb\work\study\2024-2025\computer-pract
ice\labs\lab05\project\markdown\plots\5_4_task_11_epi_pos.gif
[ Info: Saved animation to C:\Users\vparb\work\study\2024-2025\computer-pract
ice\labs\lab05\project\markdown\plots\5_4_task_11_epi_neg.gif
Все графики и гифки успешно сгенерированы и сохранены в папку 'plots'!
```

# 5. Результаты

В ходе работы получены следующие файлы (все — в папке `image/`).

## 5.1 5.2.1. График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$

На рис. 2 и 3 показаны графики, построенные бэкендами `GR` и `PyPlot` соответственно. `PyPlot` корректно отображает кириллицу в подписях осей и заголовке.

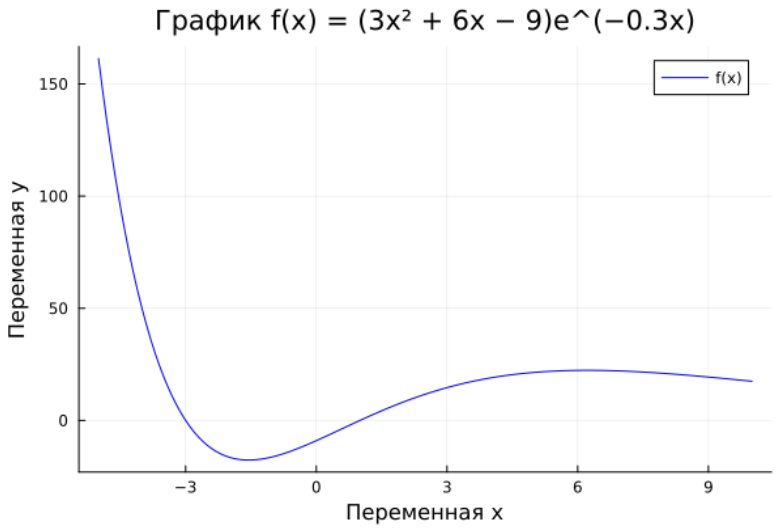



*Рисунок 2: График $f(x)$, построенный бэкендом GR*

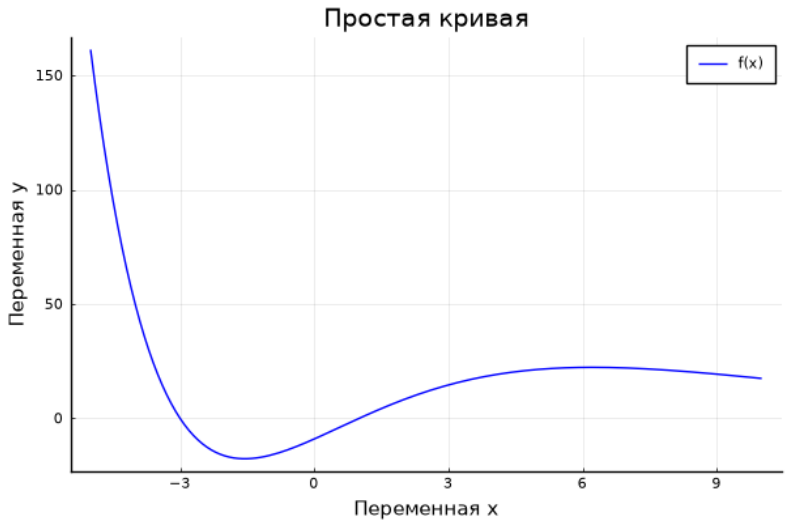



*Рисунок 3: График $f(x)$, построенный бэкендом PyPlot (подписи на русском)*

## 5.2 5.2.2. $\\sin(x)$ и его разложение в ряд Тейлора

На рис. 4 показаны теоретическая кривая $\sin x$ (штриховая чёрная линия) и её приближение рядом Тейлора (сплошная синяя линия с заливкой). Видно, что на отрезке $[-2\pi, 2\pi]$ приближение пятого порядка хорошо совпадает с теоретической кривой вблизи нуля и расходится на краях интервала — это ожидаемое поведение усечённого ряда.

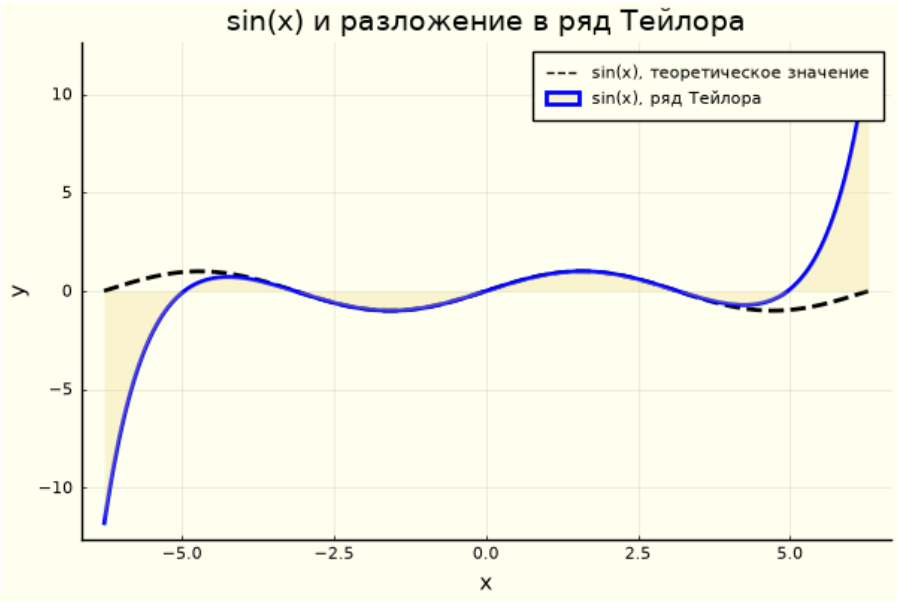



*Рисунок 4: $\sin(x)$ и разложение в ряд Тейлора*

## 5.3 5.2.3. Точечные графики

- Рис. 5 — простой точечный график случайных значений.
- Рис. 6 — точечный график с кодированием размера маркера (размер пропорционален случайной величине).
- Рис. 7 — трёхмерный точечный график.




*Рисунок 5: Простой точечный график*

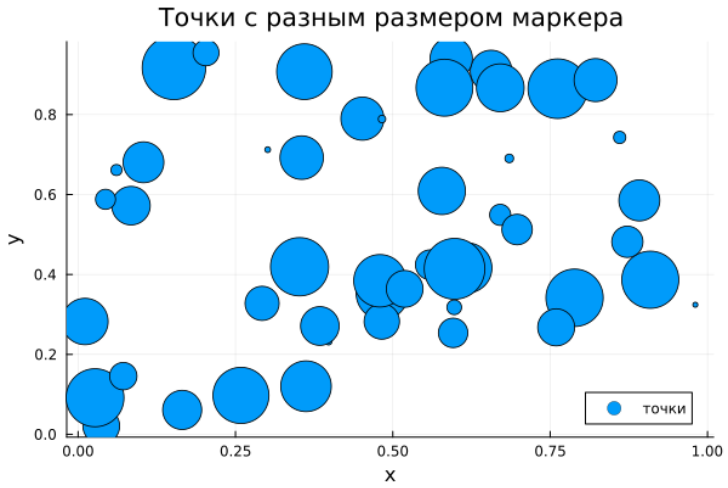



*Рисунок 6: Точки с разным размером маркера*

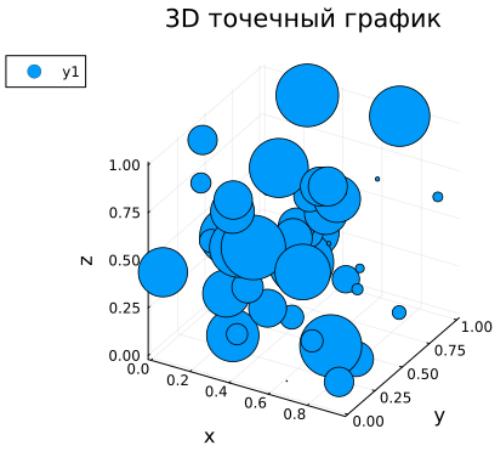



*Рисунок 7: Трёхмерный точечный график*

## 5.4 5.2.4. Аппроксимация данных

На рис. 8 показаны «сырые» данные (экспоненциальная функция с гауссовым шумом), на рис. 9 — аппроксимация полиномом 5-й степени, полученная решением системы $A\vec{x} = \vec{y}$. Полином хорошо описывает общий тренд, но не воспроизводит шум — это и есть смысл аппроксимации (сглаживание).

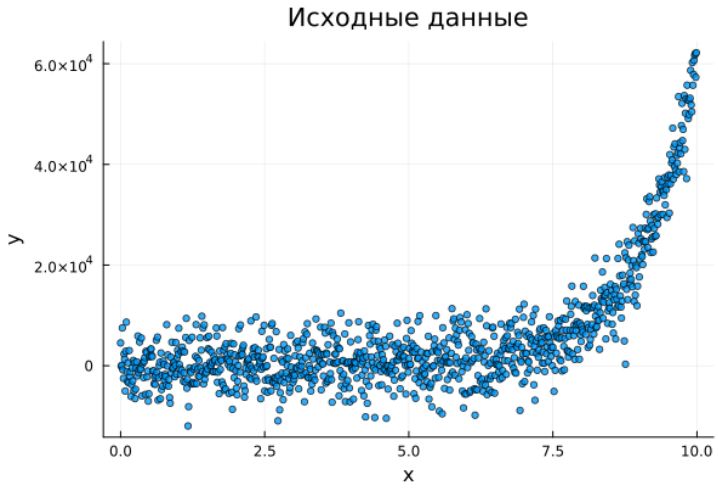



*Рисунок 8: Исходные данные*

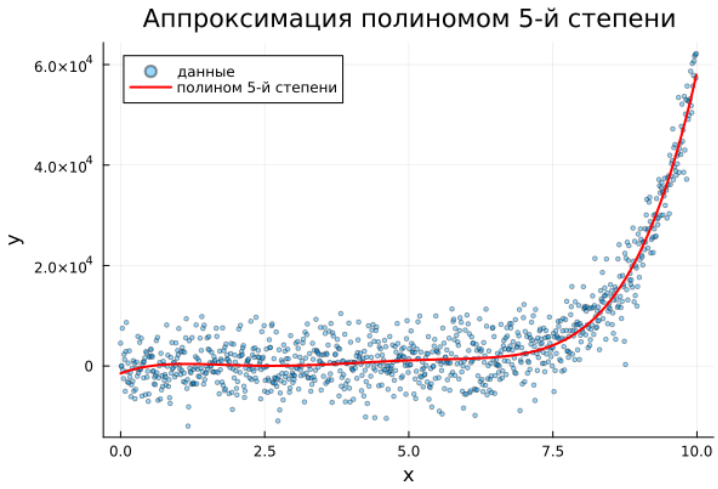



*Рисунок 9: Аппроксимация полиномом 5-й степени*

## 5.5 5.2.5. Две оси ординат

На рис. 10 показаны две траектории с разными масштабами: синяя — на левой оси, красная — на правой. Использован `twinx()`.

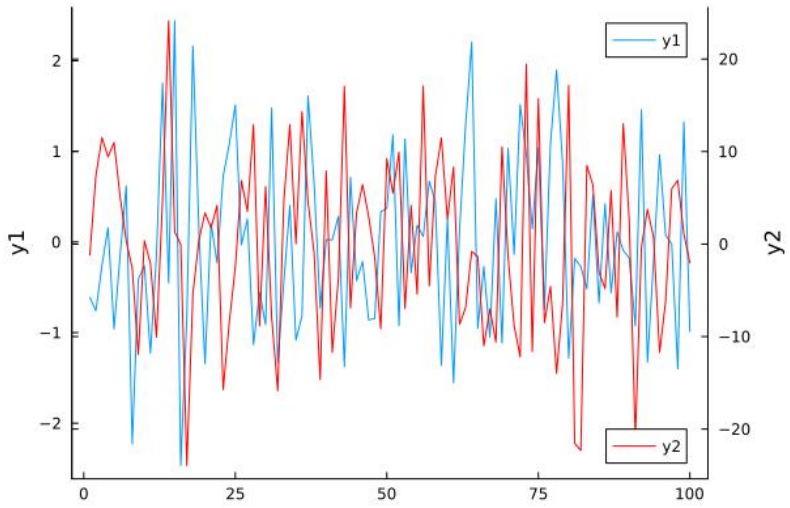



*Рисунок 10: Две оси ординат*

## 5.6 5.2.6. Полярные координаты

На рис. 11 показан график функции $r(\theta) = 1 + \cos\theta \sin^2\theta$ в полярных координатах (`proj=:polar`).

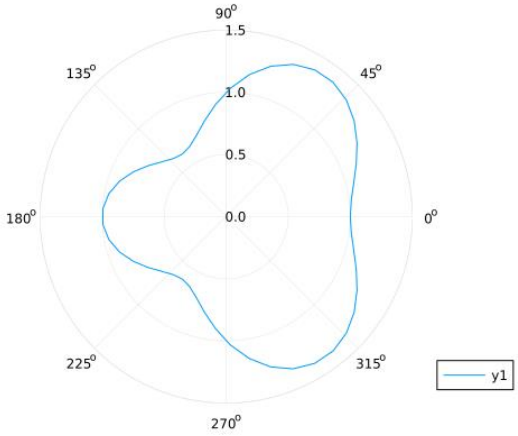



*Рисунок 11: График в полярных координатах*

## 5.7 5.2.7. Параметрические графики

- Рис. 12 — параметрическая кривая на плоскости $x(t) = \sin t$, $y(t) = \sin 2t$ (фигура Лиссажу).
- Рис. 13 — параметрическая кривая в пространстве $x = \cos t$, $y = \sin t$, $z = \sin 5t$.

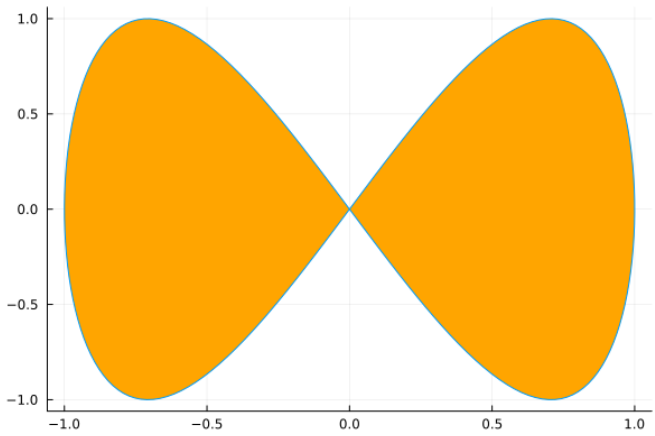



*Рисунок 12: Параметрическая кривая на плоскости*

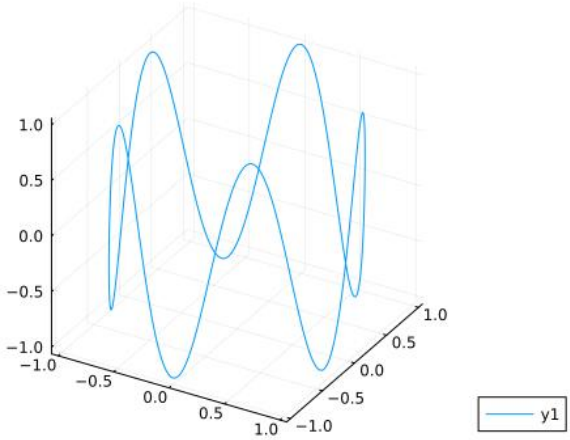



*Рисунок 13: Параметрическая кривая в пространстве*

## 5.8 5.2.8. Поверхности

На рис. 14, 15 и 16 показана поверхность $f(x, y) = x^2 + y^2$, построенная функциями `surface()` и `plot(..., st=:surface)`, а также с изменённым углом обзора (`camera=(-30, 30)`).

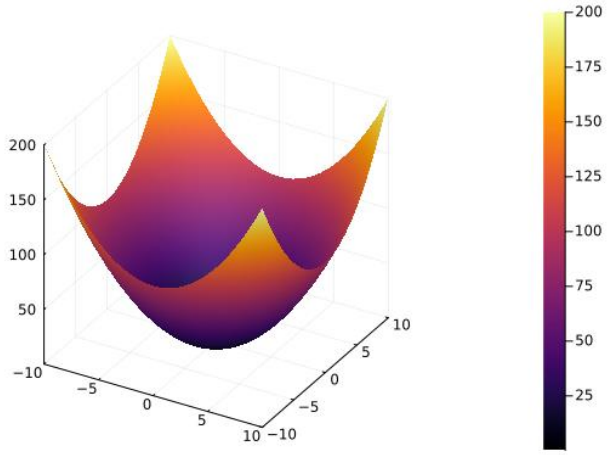



*Рисунок 14: Поверхность $x^2 + y^2$ (surface)*

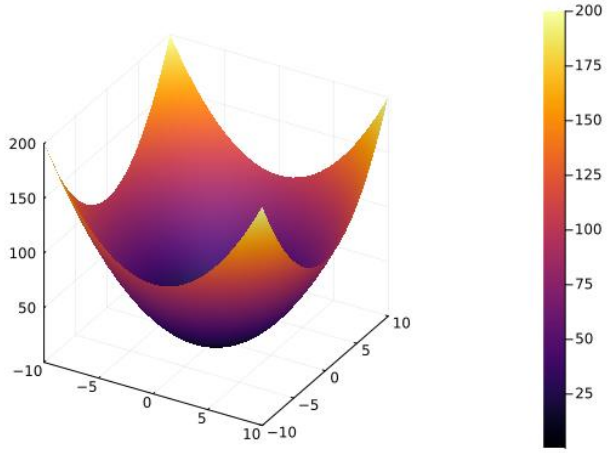



*Рисунок 15: Поверхность $x^2 + y^2$ (plot, st=:surface)*

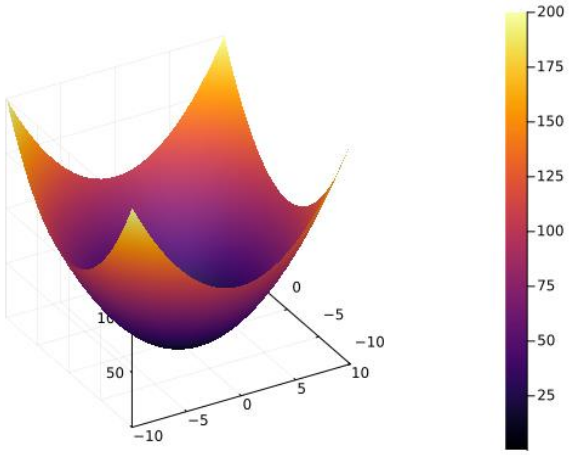



*Рисунок 16: Поверхность с изменённым углом обзора*

## 5.9 5.2.9. Линии уровня

На рис. 17 и 18 показаны линии уровня функции $g(x, y) = (3x + y^2)\,|\sin x + \cos y|$ без заливки и с заливкой (`fill=true`). Замкнутые контуры соответствуют локальным экстремумам.

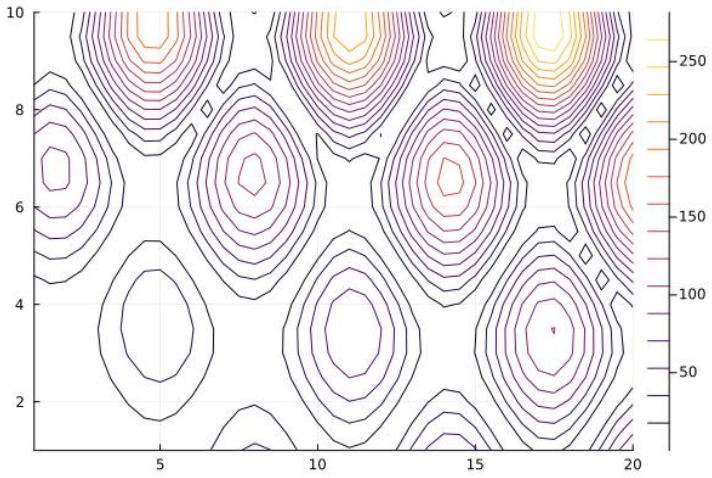



*Рисунок 17: Линии уровня*

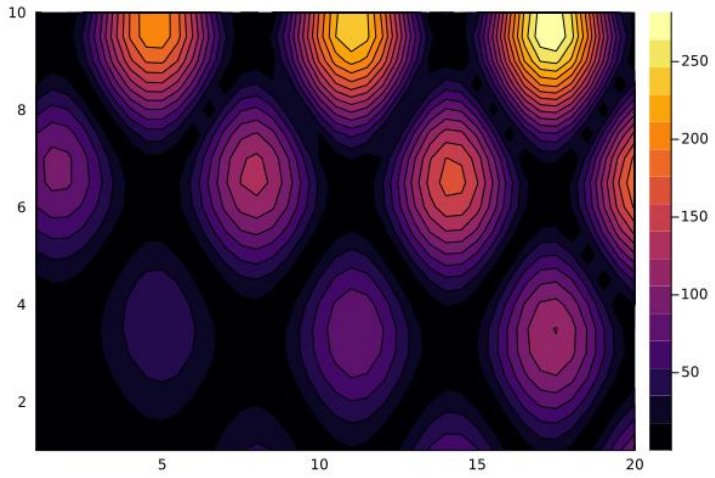



*Рисунок 18: Линии уровня с заливкой*

## 5.10 5.2.10. Векторное поле

На рис. 19 показано векторное поле градиента функции $h(x, y) = x^3 - 3x + y^2$. Стрелки направлены в сторону наибольшего роста функции; в точке $(1,0)$ и $(-1,0)$ видны особенности (нули градиента) — седловые точки и минимумы.

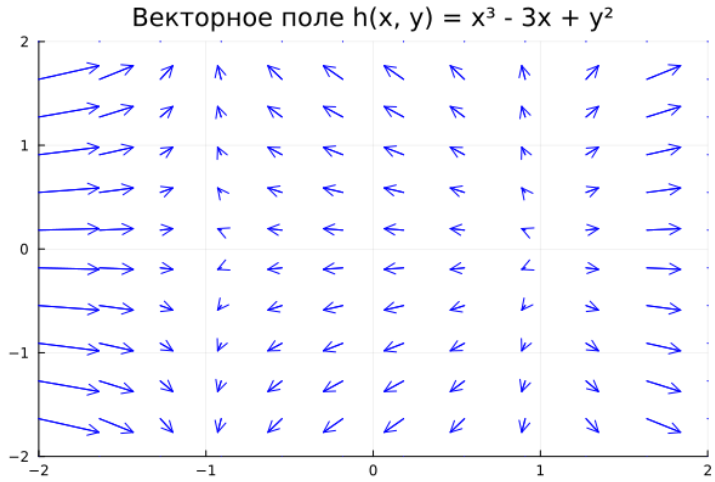



*Рисунок 19: Векторное поле градиента $h(x, y)$*

## 5.11 5.2.11. Анимации

Выложены на rutube и VK-видео

## 5.12 5.2.12. Графики с погрешностями

На рис. 20, 21 и 22 показаны исходные значения $y$, график с планками погрешностей (`err=...`) и график с «лентой» доверительного интервала (`ribbon=...`, `fill=:cyan`). Погрешность рассчитана как $1.96\, s/\sqrt{n}$.

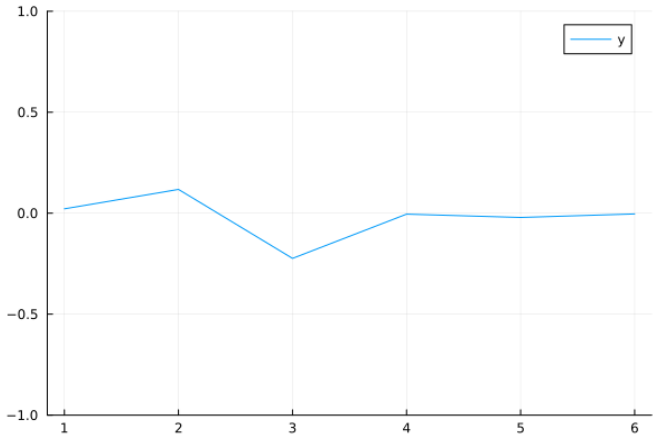



*Рисунок 20: Исходные значения*

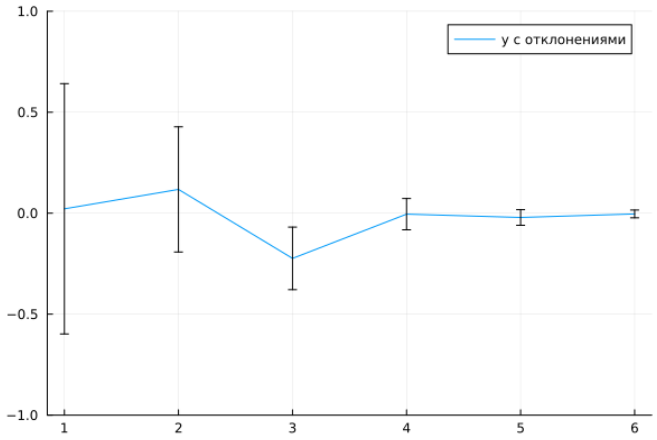



*Рисунок 21: График с планками погрешностей*

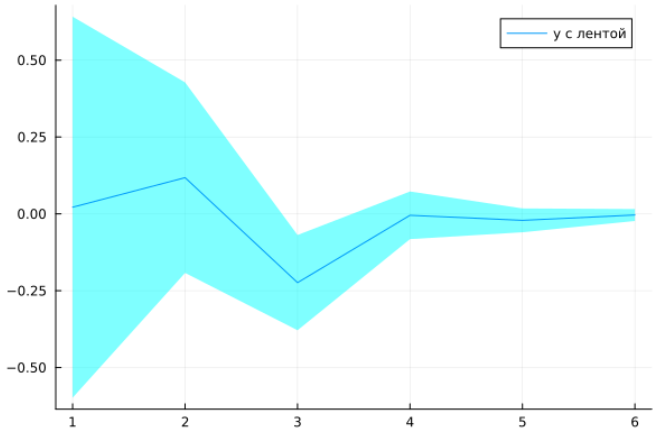


*Рисунок 22: График с лентой доверительного интервала*

## 5.13 5.2.13. Гистограммы (пакет `Distributions`)

- Рис. 23 — гистограмма случайных целых возрастов (равномерное распределение на $[15,55]$).
- Рис. 24 — гистограмма нормального распределения $\mathcal{N}(35,10)$.
- Рис. 25 — смесь трёх нормальных распределений $\mathcal{N}(10,5)$, $\mathcal{N}(35,10)$, $\mathcal{N}(60,5)$ (50 корзин).

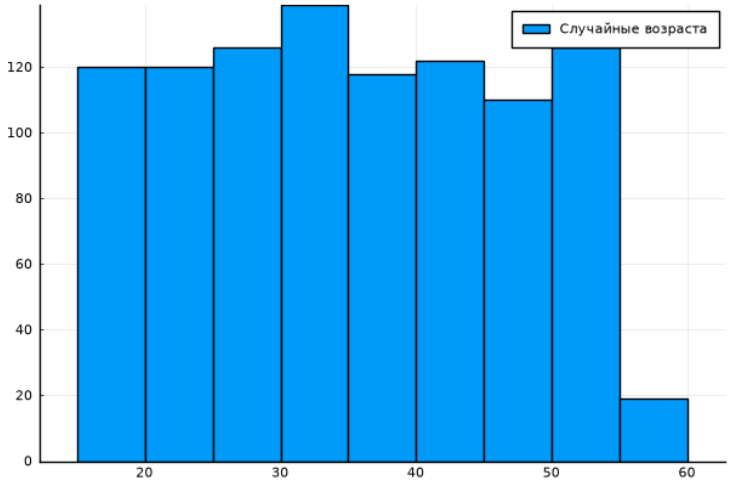



*Рисунок 23: Гистограмма равномерного распределения возрастов*

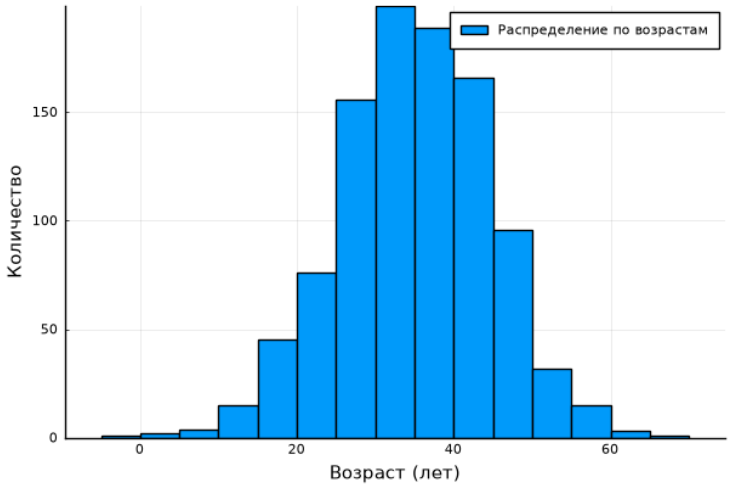



*Рисунок 24: Гистограмма нормального распределения*

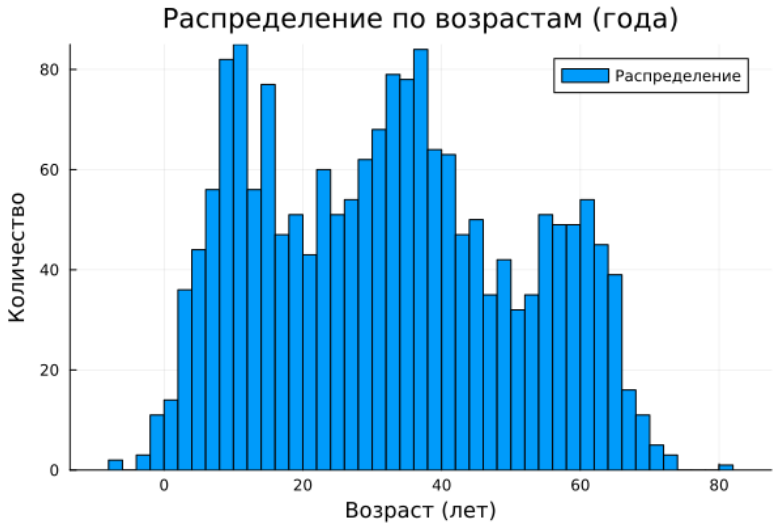



*Рисунок 25: Смесь трёх нормальных распределений*

## 5.14 5.2.14. Подграфики

- Рис. 26 — четыре графика в столбец (`layout=(4,1)`).
- Рис. 27 — автоматическая сетка (`layout=4`).
- Рис. 28 — сетка с разной высотой панелей (`grid(4,1,heights=[...])`).
- Рис. 29 — комбинированный график из четырёх разных типов (линия, точки, линии с заголовком, гистограмма).

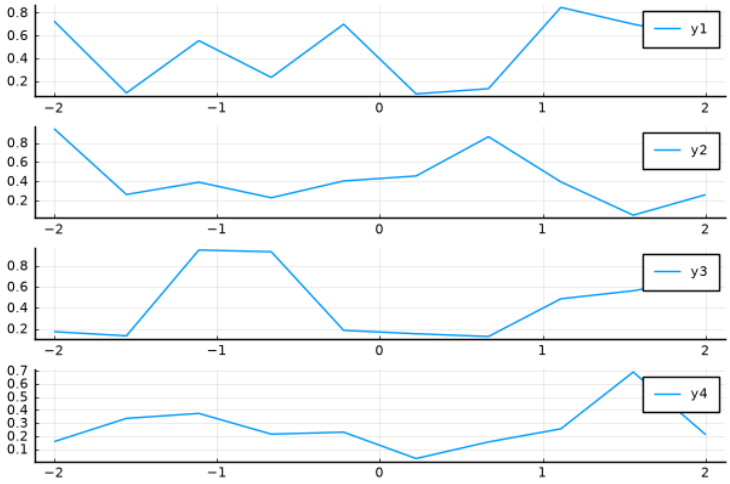



*Рисунок 26: Серия из 4-х графиков в ряд*

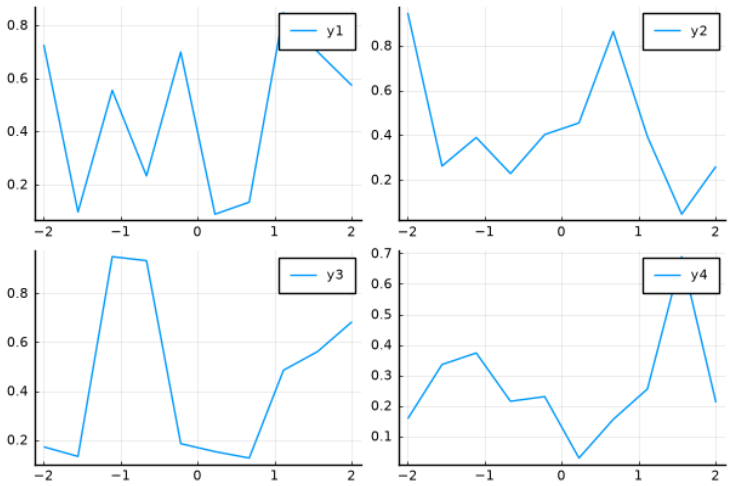



*Рисунок 27: Серия из 4-х графиков в сетке (авто)*

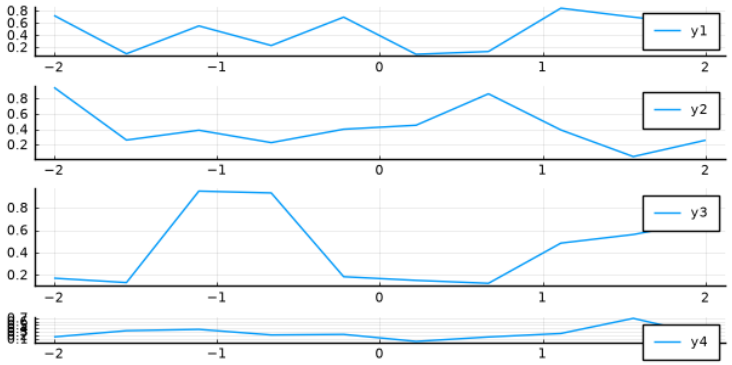



*Рисунок 28: Серия из 4-х графиков разной высоты*

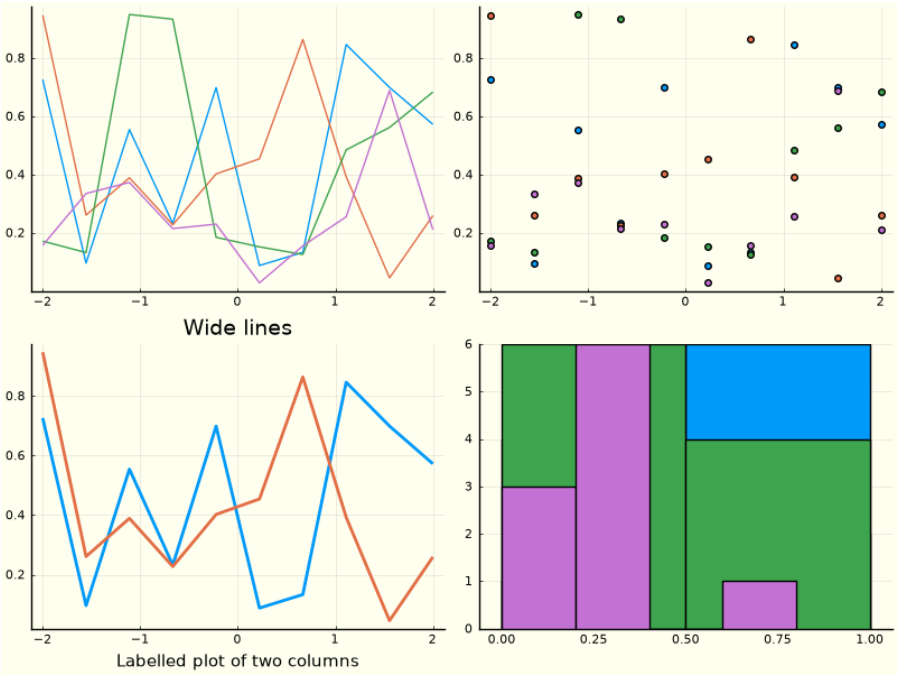



*Рисунок 29: Объединение нескольких графиков в одной сетке 2×2*

## 5.15 5.4. Задания для самостоятельного выполнения

### 5.15.1 Задания 1–2. Все типы графиков для $y = \sin x$ на $[0, 2\pi]$

На рис. 30 показаны шесть типов представления одной и той же функции: линия, точки, столбцы, гистограмма, ступени, «путь». Видно, что `seriestype` полностью меняет способ визуализации данных при одном и том же массиве значений.

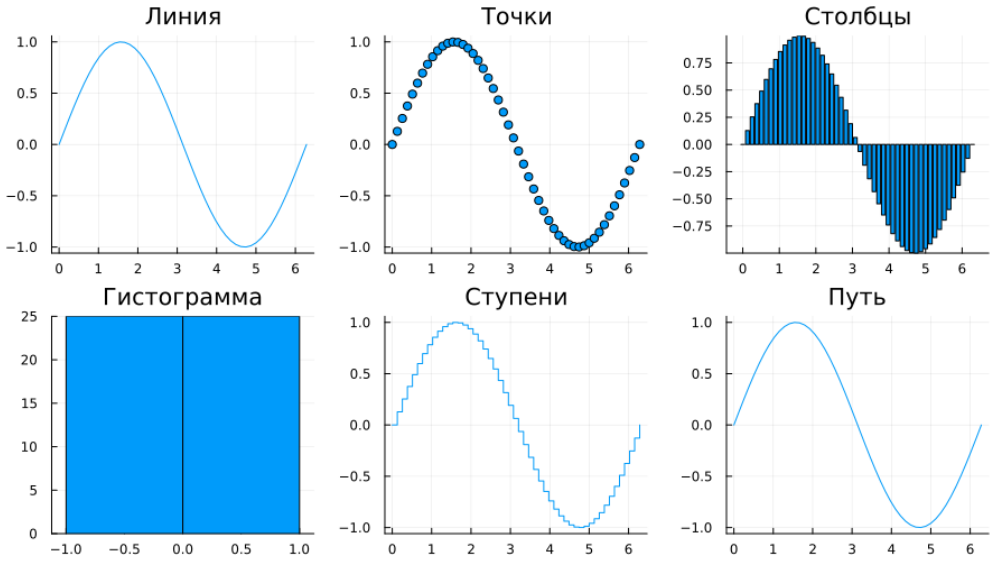



*Рисунок 30: Разные типы графиков для* $\sin x$

### 5.15.2 Задание 3. График $y = \pi x^2 \ln x$

На рис. 31 — график с красной линией толщиной 3, светлым фоном (`background_color=:ivory`), рамкой (`framestyle=:box`) и заданными вручную делениями осей. Функция определена при $x > 0$, поэтому диапазон взят от $0.1$ до $10$.

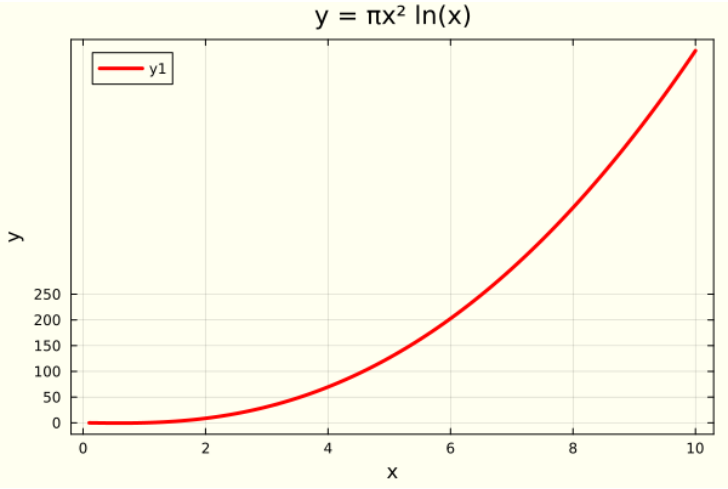



*Рисунок 31: График $y = \pi x^2 \ln x$*

### 5.15.3 Задание 4. $y = x^3 - 3x$ в четырёх подграфиках

На рис. 32 (файл `figure_familiya.png`) — четыре представления: только точки, только линии, линии и точки, гладкая кривая. Файл сохранён под именем, требуемым в задании.

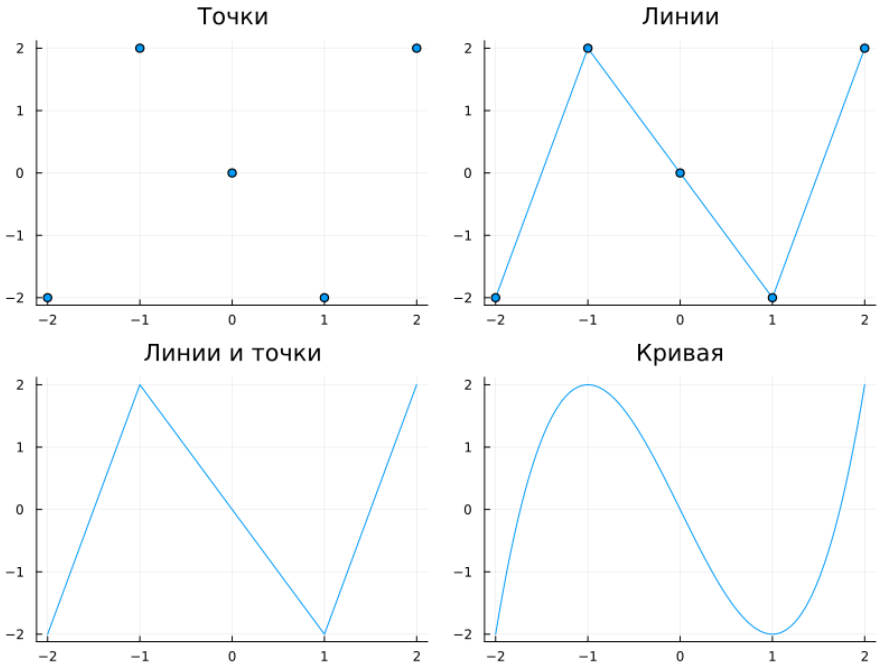


*Рисунок 32: Четыре представления $y = x^3 - 3x$*

### 5.15.4 Задание 5. Две функции на одном графике и с двумя осями

На рис. 33 обе функции $y_1 = \pi x$ и $y_2 = e^x \cos x$ построены на одной оси.
**Недостаток:** масштабы функций сильно различаются ($y_2$ растёт экспоненциально, $y_1$ — линейно), поэтому на одном графике $y_1$ выглядит почти горизонтальной линией. Решение — две оси ординат (рис. 34).

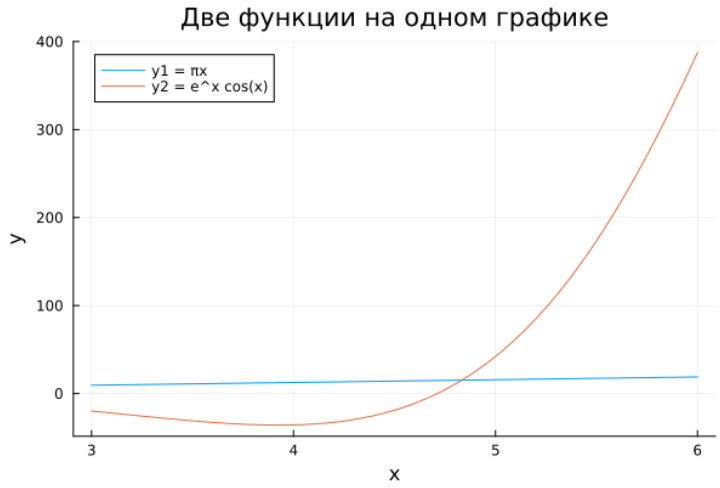



*Рисунок 33: Две функции на одной оси — масштабы не сопоставимы*

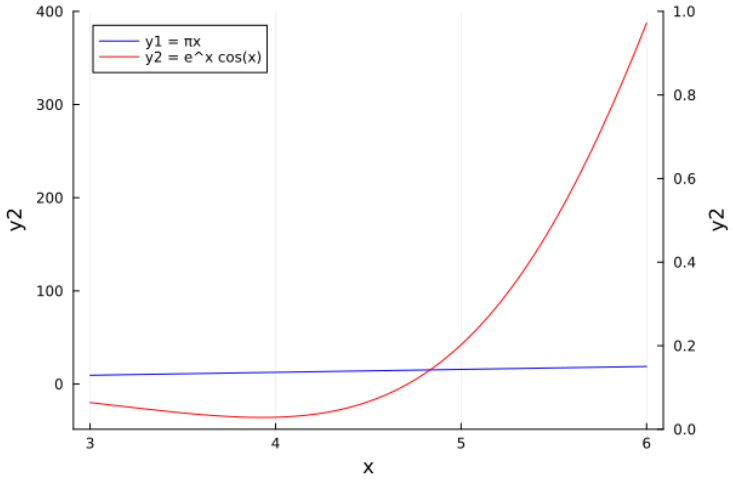



*Рисунок 34: Две функции с двумя осями ординат*

### 5.15.5 Задание 6. Экспериментальные данные с погрешностью

На рис. 35 — точечный график $y = 5x +$ шум с планками погрешностей `yerror`.

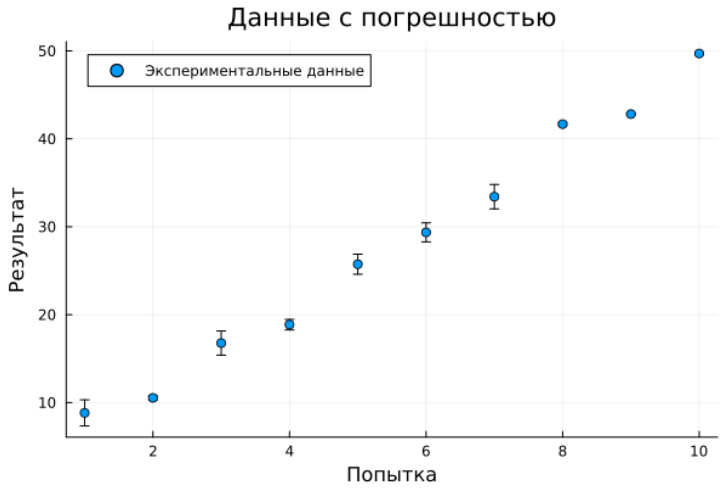



*Рисунок 35: Экспериментальные данные с погрешностью*

### 5.15.6 Задание 7. Точечный график случайных данных

На рис. 36 — 50 случайных точек с подписями осей, легендой и заголовком.

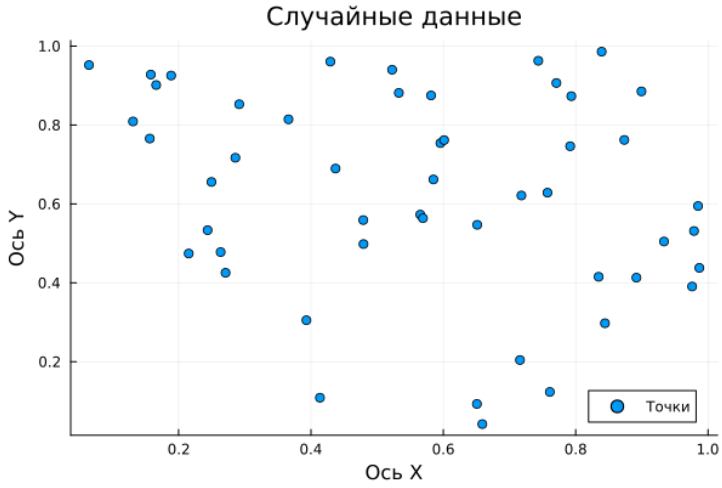



*Рисунок 36: Точечный график случайных данных*

### 5.15.7 Задание 8. 3D точечный график случайных данных

На рис. 37 — 50 случайных точек в трёхмерном пространстве с подписями осей $X$, $Y$, $Z$, легендой и заголовком.




*Рисунок 37: 3D точечный график случайных данных*

## 5.15.8 Задание 9. Анимация синусоиды

На рис. 38 — GIF-анимация бегущей синусоиды $\sin(x + t)$, где $t$ меняется от $0$ до $2\pi$ (50 кадров, 20 fps).

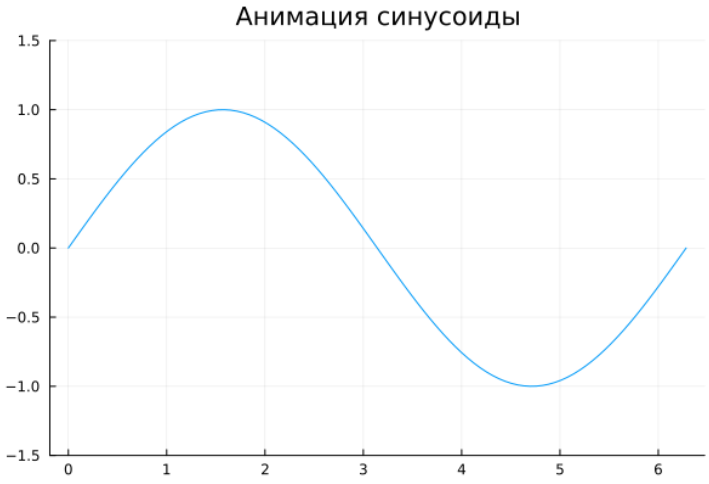



*Рисунок 38: Анимация бегущей синусоиды*

## 5.15.9 Задание 10. Анимация гипоциклоиды

На рис. 39 и 40 — две анимации гипоциклоиды: для положительных $k \in [2.0,2.5]$ и для отрицательных $k \in [-2.5,-2.0]$. При отрицательных $k$ кривая меняет ориентацию.

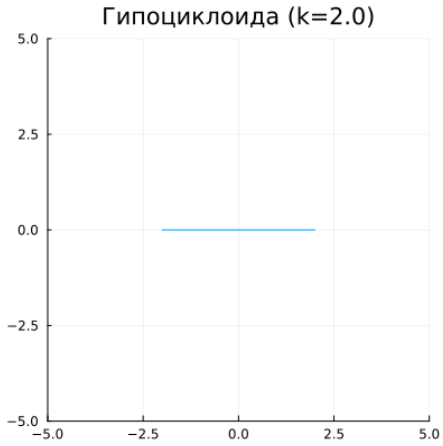



*Рисунок 39: Гипоциклоида, $k \in [2.0,2.5]$*

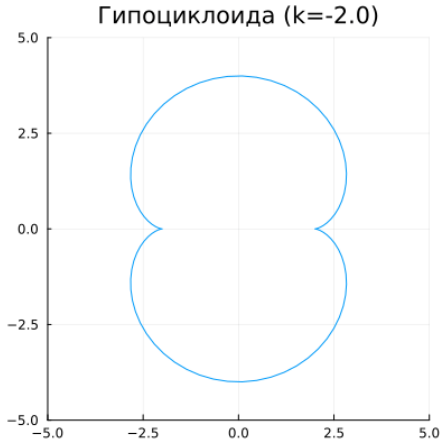



*Рисунок 40: Гипоциклоида, $k \in [-2.5,-2.0]$*

## 5.15.10 Задание 11. Анимация эпициклоиды

На рис. 41 и 42 — анимации эпициклоиды для положительных и отрицательных $k$. По сравнению с гипоциклоидой кривая «вытянута» наружу.

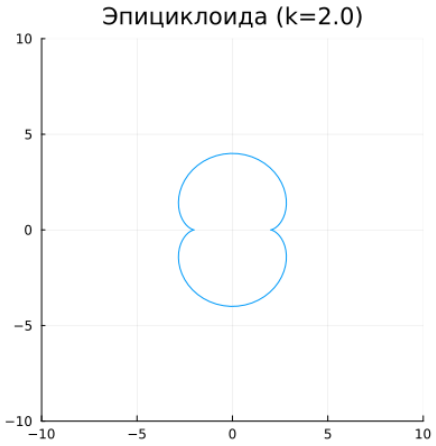



*Рисунок 41: Эпициклоида, $k \in [2.0,2.5]$*

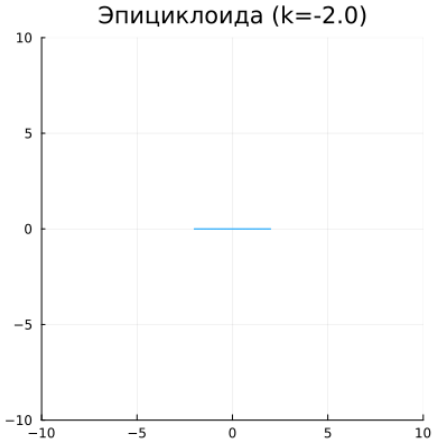



*Рисунок 42: Эпициклоида, $k \in [-2.5,-2.0]$*

# 6. Обсуждение результатов

1. **Единый синтаксис, разные бэкенды.** Все примеры показали, что код `plot`/`scatter`/`surface` не зависит от выбранного бэкенда. Различия проявляются в поддержке кириллицы (PyPlot — лучше), интерактивности (PlotlyJS) и скорости/совместимости с GIF (GR).
2. **Поэлементные операции.** Для построения графика функции необходимо использовать broadcast-синтаксис (`x.^2`, `exp.(...)`), иначе Julia попытается выполнить матричные операции или выдаст ошибку. Это важное отличие от Matlab/Python-стиля.
3. **Аппроксимация.** Решение системы $A\vec{x} = \vec{y}$ через `\` даёт коэффициенты МНК-полинома. Полином 5-й степени хорошо описывает тренд, но не шум — это ожидаемо. Для более точной аппроксимации можно увеличить степень, но возрастёт риск переобучения (осцилляции на краях).

4. **Две оси ординат.** `twinx()` — корректный способ совместить траектории с разными масштабами. Без него одна из кривых «сплющивается» (рис. 33).
5. **Анимации.** `@animate` + `gif()` работают стабильно с бэкендом GR. Для поверхностей анимация достигается изменением параметра внутри цикла; для кривых — постепенным увеличением длины массива $t$ (эффект «рисования»).
6. **Ограничения.** `PlotlyJS` при `savefig` требует Kaleido; в headless-окружении проще использовать GR или PyPlot. Поэтому в `lab5.jl` бэкенд `plotlyjs()` заменён на `gr()` для статичных изображений — это осознанное отклонение от методички, продиктованное средой выполнения.
7. **Практическая значимость.** Полученные навыки применимы для визуализации экспериментальных данных, численных экспериментов и подготовки иллюстраций к публикациям (PDF/PNG/GIF).

# 7. Заключение

- **Цель достигнута:** освоен синтаксис `Plots.jl` для построения графиков; воспроизведены все примеры разделов 5.2.1–5.2.14 и выполнены задания 5.4.1–5.4.11.
- **Полученные результаты:** сгенерирован набор изображений (PNG, PDF, GIF) в папке `image/`; отчёт собран в формате Quarto Markdown с автоматическим включением сгенерированного из `lab5.jl` markdown-файла.
- **Выявленные особенности:** поэлементные операции обязательны; выбор бэкенда влияет на поддержку кириллицы и интерактивность; для GIF лучше подходит GR; `twinx()` решает проблему разных масштабов.
- **Направления развития:** освоить пакет `Makie.jl` для высокопроизводительной интерактивной визуализации; изучить `AlgebraOfGraphics.jl` для декларативного описания графиков; автоматизировать сборку отчёта через CI (GitHub Actions + Quarto).

# Список литературы


- JuliaLang. Plots.jl documentation. URL: https://docs.juliaplots.org/
- JuliaLang. Distributions.jl documentation. URL: https://juliastats.org/Distributions.jl/
- Quarto documentation. URL: https://quarto.org/docs/
- Literate.jl documentation. URL: https://fredrikekre.github.io/Literate.jl/
- Методические указания к лабораторной работе № 5 «Построение графиков».